Technical University of Munich
School of Engineering and Design
Prof. Dr. Martin Werner

Principles of Programming
(WiSe XX/XX)
Tutorial 1-40

Tasks marked with a star like **Optional Task*** are optional. Tasks marked like **Hard Task+** are given, but it is not expected that you solve them now. It is great if you learn to solve them during the lecture. Go back to them after a few weeks and see your own progress.

Learning Outcome: Basic Computing

1 Python Environment

Task 1: Environments and Jupyter Notebooks

Create an environment and start a Jupyter Notebook:

```
python -m venv /path/to/my/envirnoment
source /path/to/my/envirnoment/bin/activate
pip install notebook
jupyter notebook
```

You can exit the environment by typing *deactivate* in your shell. If you want to use Anaconda, you may do this inside the "Anaconda Navigator" or execute the following lines:

```
conda create --name myfirstenv
conda activate myfirstenv
conda install notebook
jupyter notebook
```

You can exit the conda environment by typing *conda deactivate* in your shell.

Task 2: Basic Python Exercise

Solve the following tasks inside a Jupyter Notebook. An example follows each task.

a. print "Hello, World"

```
Output: Hello, World
```

b. multiplicate two numbers

```
Input: 53, 834
Output: 44679
```

c. remove first n characters from a string

```
Input: "Hello", 2
Output: llo
```

d. check if the first and last number of a list are the same

```
Input: [5, 3, 1, 3, 4, 5]
Output: True
```

e. return the count of a given substring from a given string

```
Input: "I have no idea what I am doing.", "I"
Output: 2
```

f. check if a given number is a palindrome number

```
Input: 32123
Output: True
```

Task 3: Python Scripts

Install a code editor (e.g., VS Code) and create a Python script (e.g., tutorial_1.py). Implement Task 2 a. - f. with a main function and run it. The basic structure of your script may look like the following:

```
def main():
    # Your code here ...
    return result

if __name__=="__main__":
    main()
```

Task 4: Command Line Python

Run your Python script from Task 3 in the command prompt (Windows) or terminal (Linux) by typing *python /path/to/tutorial_1.py*. Then, do Task 2 a. - b. without a script by running Python directly in the shell. You can start Python from the command line by typing *python*.

2 Python Loops

Task 5: For Loops

Use a for loop to print a multiplication table from 1 to 10.

Expected output:

```
1   2  3  4  5  6  7  8  9  10
2   4  6  8 10 12 14 16 18  20
3   6  9 12 15 18 21 24 27  30
4   8 12 16 20 24 28 32 36  40
5  10 15 20 25 30 35 40 45  50
6  12 18 24 30 36 42 48 54  60
7  14 21 28 35 42 49 56 63  70
8  16 24 32 40 48 56 64 72  80
9  18 27 36 45 54 63 72 81  90
10 20 30 40 50 60 70 80 90 100
```

Task 6: While Loops

Using a while statement, write a function that takes two int values (”base” and ”exponent”) as input and prints the base to the power of the exponent.

Example
Expected output for input base=2 and exp=5:

```
2 to the power of 5 is:  32
```

3 Python List Comprehensions

Solve the following tasks with list comprehensions:

```
newlist = [expression for item in iterable if condition]
```

Task 7: Reverse String

Write a function that reverses a string.

Example
Expected output for input s = ”hello”:

```
["o","l","l","e","h"]
```

Task 8: Sort Array

Given an integer array nums, move all the even integers at the beginning of the array followed by all the odd integers. Return any array that satisfies this condition.

Example
Expected output for input nums = [3,1,2,4]:

```
[2,4,3,1]
```

Explanation: The outputs [4,2,3,1], [2,4,1,3], and [4,2,1,3] would also be accepted.

Task 9: Fizz Buzz

Fizz buzz is a group word game for children to teach them about division. Players take turns to count incrementally, replacing any number divisible by three with the word "fizz", and any number divisible by five with the word "buzz", and any number divisible by both 3 and 5 with the word "fizzbuzz" (from Wikipedia).

Given an integer n, return a string array answer (1-indexed) where:

a. answer[i] == "FizzBuzz" if i is divisible by 3 and 5.

b. answer[i] == "Fizz" if i is divisible by 3.

c. answer[i] == "Buzz" if i is divisible by 5.

d. answer[i] == i (as a string) if none of the above conditions are true.

Example
Expected output for input n = 15:

```
["1","2","Fizz","4","Buzz","Fizz","7","8","Fizz","Buzz","11","Fizz
    ","13","14","FizzBuzz"]
```

Task 10: Numpy

Import the NumPy package (Official documentation: https://numpy.org/doc/) and use it to solve the following tasks:

a. Print the Numpy version and configuration

b. Create a 10x10 matrix filled with zeros

c. Find the shape of an arbitrary array (e.g. the one from a.)

d. Create a 4x6 matrix with zeros but the value at index [2, 3] which is 1

e. Create a 3x7 matrix filled with values ranging from 21 to 42

f. Create a vector of size 10 filled with random values from 0 to 1

g. Reverse the vector from f. (first element becomes last)

h. Create a 3x3 matrix with random integer values ranging from 0 to 8

i. Find indices of non-zero elements from [1,0,7,0,1,0,0,1,0,0,1]

j. Find all unique values from [1,0,7,0,1,0,0,1,0,0,1]

k. Create a 3x3 identity matrix

l. Multiply a 5x3 matrix by a 3x2 matrix

m. Create a random vector of size 10 and sort it

n. Create a 5x5x5 array with random values and find the minimum, the maximum and the mean value

4 Matplotlib

Task 11: Matplotlib

Import the Pyplot package from Matplotlib (Official documentation: https://matplotlib.org/stable/api/index.html) and use it to solve the following tasks:

a. Load an image of your choice

b. Display the image

c. What are the spatial dimensions, data type, and range of the values of the image?

d. What is the color value of the pixel at position [21,42]?

e. Calculate the mean colour value for each channel over all pixels

f. Convert the image in grayscale and display it. The gray value h can be calculated from the RGB values using the formular h = 0.2989r + 0.5870g + 0.1140b.

g. Change the colormap so that the image is actually shown as grayscale and display it again

h. Save the grayscale image in the PNG-format

i. Create and plot a histogram of the grayscale image

Task 12: Numpy continued

Use Numpy to solve the following tasks.

a. Create a 2D array with 1 on the border and 0 inside

b. Normalize a 5x5 random matrix

c. Multiply a 4x3 matrix by a 3x5 matrix

d. Create a random vector of size 6 and replace the maximum value by 0

e. Create a 2D integer matrix and return the transpose

f. Create a checkerboard 10x10 matrix using the "tile" function

Task 13: Matplotlib continued

Use Matplotlib to solve the following tasks. First, we create some random dataframe with pandas:

```
import pandas
df = pandas.DataFrame(data={"a":np.random.randint(0, 100, 50),
                            "b":np.random.randint(0, 100, 50),
                            "c":np.random.randint(0, 100, 50)})
```

a. Create a figure of size 15x8 with two subplots, top and bottom

b. Draw two lines (data a and b) in the top plot, one blue and one orange, and add a grid

c. Create two vectors of size 50 filled with random values in range 0-10 and use "matplotlib.pyplot.fill_between" function to plot an artificial standard deviation for both lines (Hint: Set "alpha=0.2" and "edgecolor=None")

d. For the top plot, add a title and a legend in the top-middle

e. In the bottom plot, create a graph of only the data points (c), marked by circles, but with no line connecting the points. Further, add a grid, title, and legend for the bottom plot

a. Create a figure of size (15,18) with three subplots, aligned vertically

b. Create bar graphs for data a and data b in the first two subplots but in third subplot put data b on top of data a

c. Add a legend and title for each plot

d. Plot the magnitude of each bar on top of it

5 Problem solving

The following problems are meant to be solved with pure Python and do not require any additional packages. You must recall all the previously discussed concepts (lists, loops, conditions, etc.) to solve them. However, you may optionally also try to solve them with any other programming language of your choice.

Task 14: Single Number

Given a non-empty array of integers `nums`, every element appears twice except for one. Find that single one.

Example 1:

```
Input: nums = [2,2,1]
Output: 1
```

Example 2:

```
Input: nums = [4,1,2,1,2]
Output: 4
```

Example 3:

```
Input: nums = [1]
Output: 1
```

Task 15: Sum Multiples

Given a positive integer `n`, find the sum of all integers in the range $[1, n]$ inclusive that are divisible by 3, 5, or 7. Return an integer denoting the sum of all numbers in the given range satisfying the constraint.

Example 1:

```
Input: n = 7
Output: 21
```

Explanation: Numbers in the range [1, 7] that are divisible by 3, 5, or 7 are 3, 5, 6, 7. The sum of these numbers is 21.

Example 2:

```
Input: n = 10
Output: 40
```

Explanation: Numbers in the range [1, 10] that are divisible by 3, 5, or 7 are 3, 5, 6, 7, 9, 10. The sum of these numbers is 40.

Example 3:

```
Input: n = 9
Output: 30
```

Explanation: Numbers in the range [1, 9] that are divisible by 3, 5, or 7 are 3, 5, 6, 7, 9. The sum of these numbers is 30.

Task 16: Count Prefixes

You are given a string array `words` and a string s, where every `words[i]` and `s` comprise only lowercase English letters. Return the number of strings in words that are a prefix of `s`. A prefix of a string is a substring that occurs at the beginning of the string. A substring is a contiguous sequence of characters within a string.

Example 1:

```
Input: words = ["a","b","c","ab","bc","abc"], s = "abc"
Output: 3
```

Explanation: The strings in words which are a prefix of `s` = ”abc” are: ”a”, ”ab”, and ”abc”. Thus the number of strings in words which are a prefix of `s` is 3.

Example 2:

```
Input: words = ["a","a"], s = "aa"
Output: 2
```

Explanation: Both strings are a prefix of `s`. Note that the same string can occur multiple times in words, and it should be counted each time.

6 Repetition, Turtle graphics, recursion

Python has a built-in module named `turtle`, similar to "Niki the robot". Turtle graphics provide a visualization of a "turtle" with a pen that draws on a sheet of paper on the floor. It was designed as an educational tool and is an effective way for learners to encounter programming concepts.

To start, get `turtle` running on your system. You find a comprehensive guide for `turtle` here (Hint: In case the `No module named '_tkinter'` error is not resolved by installing the `tk` package, run `sudo apt-get install -y python3-wxgtk4.0`).

Before solving the following tasks, familiarize yourself with Turtle graphics and try different methods described in the reference.

Generally, all tasks are meant to repeat the programming concepts we learned in the previous lessons. Try to write efficient and clean code.

Task 17: Turtle drawing

Write functions that take a single parameter `size` for the edge length as input to create the given shapes with turtle graphics.

- Triangle
- Square
- Hexagon
- Octagon

Task 18: Turtle drawing continued

Try to write a script to recreate the following figure (or similar).

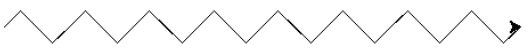


Figure 1: Rectangle pattern.

Next, experiment a bit and implement your own ideas. Following is another example of a pattern created from circles. Be creative!

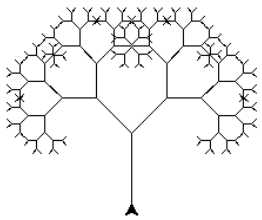


Figure 2: Circle pattern.

Task 19: Recursive spiral

Implement a recursive function `spiral(int: size, float: angle, float: scale, int: maxSize)` that draws spirals like the example below.

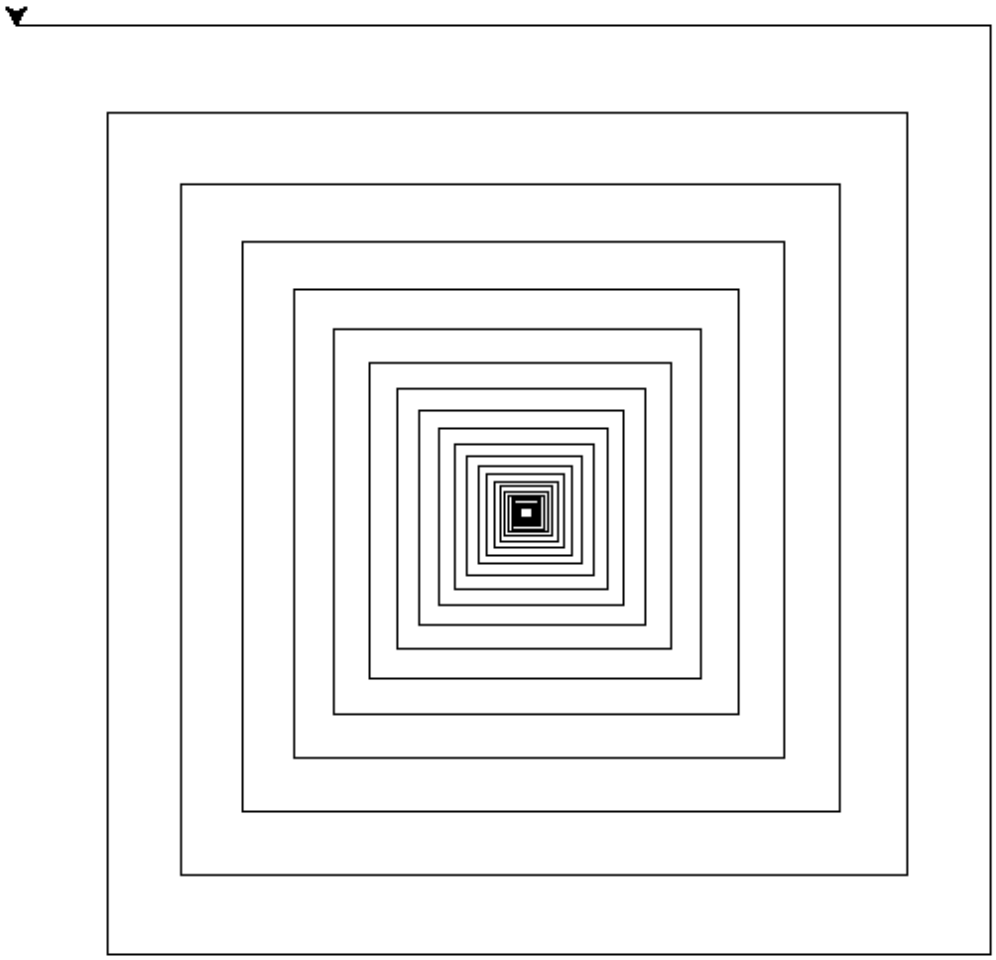


Figure 3: A spiral.

Task 20: Recursive tree

Implement a recursive function `tree(int: branches, int: size, float: angle, float: scale)` that draws trees like the examples below.

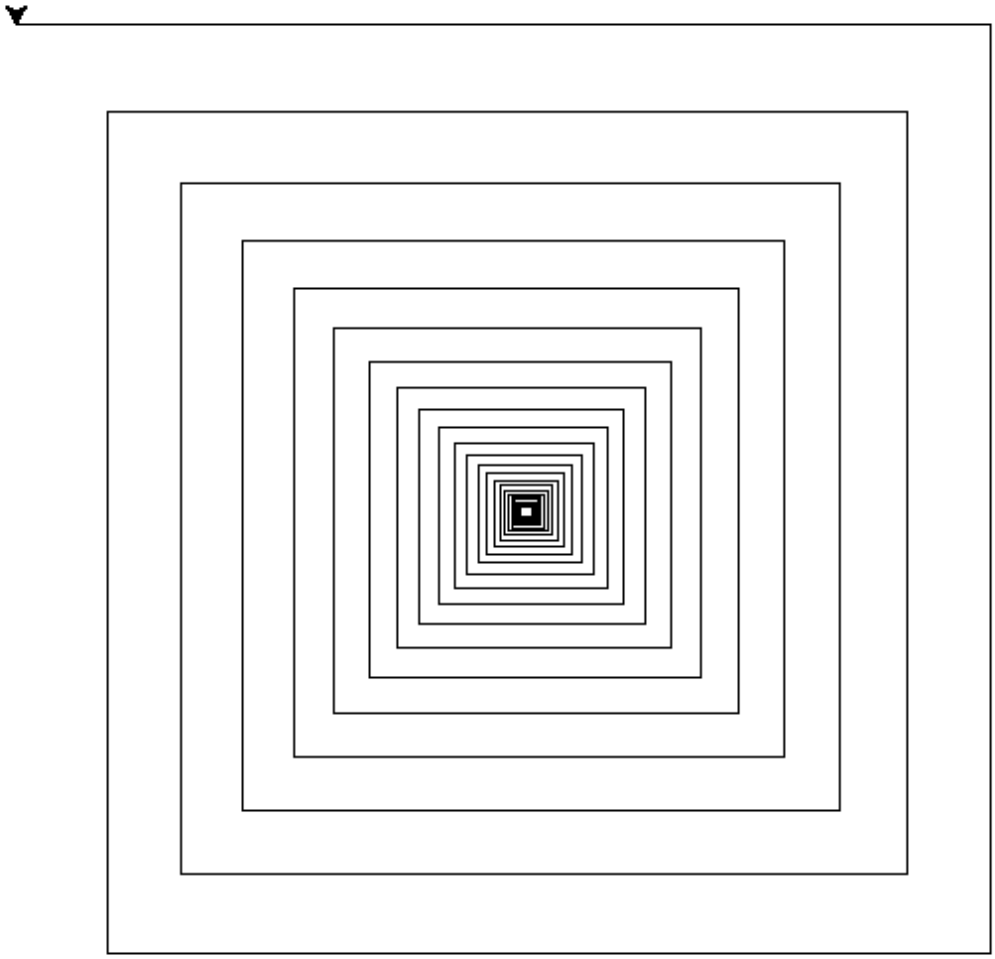


Figure 4: A tree created with 45° angles.

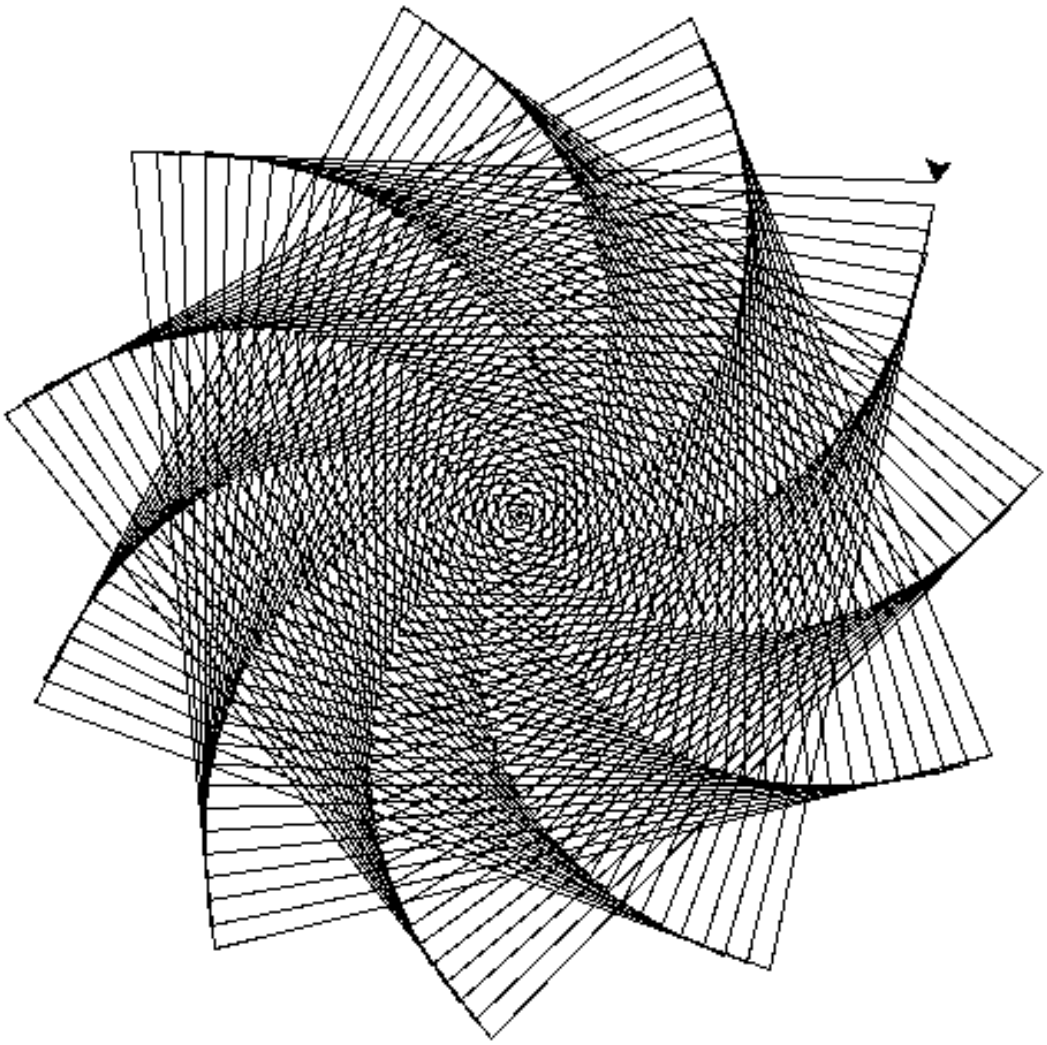


Figure 5: A tree created with 90° angles.

7 Douglas Peucker Algorithm, Recursion

Task 21: The Douglas-Peucker Algorithm

The Douglas-Peucker algorithm (also Ramer-Douglas-Peucker algorithm) is an algorithm that decimates a curve composed of line segments to a similar curve with fewer points. It was one of the first efficient algorithms for cartographic generalization. First, find a suitable source and familiarize yourself with the algorithm.

Task 22: Implement the Douglas-Peucker Algorithm

Implement the Douglas-Peucker algorithm to simplify trajectories. Test it on a simple artificial trajectory and visualize the original and simplified trajectory with `matplotlib`. For example, your trajectory for testing may look like this:

```
T = [[1,1],[2,3],[3,4],[5,1],[6,2],[8,3],[10,5]]
```

Additional Information:

- As input, the Douglas-Peuker algorithm is given a trajectory as a sequence of points (timestamps are not needed) and an error threshold epsilon.
- It starts by trying to approximate the trajectory with the shortest trajectory possible, containing only the start and end points.
- It proceeds from such shortest trajectories by finding the point of largest error and adding the point of the original trajectory into the current simplification until this error is smaller than epsilon.
- The resulting trajectory is the simplification.

Implementation Hints:

- You need to implement a point line distance function.
- You can implement the simplification using a property map approach: An array of boolean values of the same size as the trajectory can be updated if a point needs to be inserted.
- It makes sense to have a findMaxError function returning the size of the error and the location.
- As a variation, you can replace the epsilon with a point budget like "simplify as long as the number of points is smaller than my budget".

Task 23: Apply the Douglas-Peucker Algorithm

Apply the algorithm to a large polygon and test different values for epsilon. It is suggested to take, for example, the administrative border of a country of your choice (you can download shapefiles for most countries here), but you may also use any other large polygon. You can read shapefiles with the PyShp package. Below is a way to read in a shapefile with PyShp.

```
import shapefile as shp

sf = shp.Reader(file)

x = []
y = []
for shape in sf.shapeRecords():
    x.append([i[0] for i in shape.shape.points[:]])
    y.append([i[1] for i in shape.shape.points[:]])
```

Finally, think about ways to optimize your code.

8 Solving complex computational problems Pt1

In this longer tutorial, we want to jointly look at how we can combine our initial knowledge to build a full deep learning system in which a large spatial image and GDAL are used together with Tensorflow to predict a certain spatial feature.

In this first part, we will build knowledge on GDAL, cut out small images (called tiles or patches) from your measured data, and assign a class to it.

Task 24: Data exploration

Initially, it is proposed to build knowledge on GDAL and understand the provided data `germany.tiff`. Therefore, go through the script `GDAL.ipynb`, then try to adapt it. Choose a point of interest (POI) and extract the area around it as a JPG file for further processing. Finally, you should end up with an RGB image file `germany_poi.jpg`.

Task 25: Create tiles

Tile the file `germany_poi.jpg` from Task 1 by writing a function `tile()`. It is suggested to create tiles/patches with a size of 48×48 or 64×64, but you may choose any size you like. Also, try to overlap the tiles by a small amount. Finally, save the tiles in a new directory.

Task 26: Ground truth

Before we can run a deep learning model, we need ground truth data. You may use any ground truth for any spatial feature you like. However, here, we will use, as an example, forest as a spatial feature and download it via the Overpass API. Overpass-turbo is a web-based data mining tool for Open Street Map and can be accessed here. You can read the documentation on the API and how to write queries here. Then, we import the extracted geojson file with QGIS or ArcGIS, rasterize the data, and export it as TIFF (in QGIS click "Raster", "Conversion", then "Rasterize"). It is important to extract the TIFF with the right coordinate system and bounding box applied.

Task 27: Label the data

Finally, write a function `label_data()` to assign a class to the tiles. Here, the simplest approach is to extract a random patch (random location, but within the bounds of the image), and if it contains forest (`np.sum(...) != 0`), you store a JPG file of the patch in a directory `no_forest`. Otherwise, put it into `forest`. You end up with two directories of images.

9 Solving complex computational problems Pt2

In this second part, we will learn about Keras data loaders and train a neural network to detect some spatial features. It is proposed that knowledge on Keras be first built with a tutorial for a simple deep-learning example. A beginner-friendly overview can be found here. Note that for multidimensional raster data, you may want to use convolutional layers like shown here. You may even extend the example with RGB images already (you could build an RGB image from three identical grayscale layers).

Task 28: Data loading

A convenient way to load and prepare your data for deep learning with Keras is a "data loader". You can read the official documentation and examples here and subsequently here. Set up data loaders for train, test, and validation data.

Task 29: Deep learning model

Build and compile a deep learning model with Keras. You may adapt the convolutional neural network from this example.

Task 30: Training and evaluation

Finally, train (`model.fit()`) and evaluate (`model.evaluate()`) your model. Plot some examples for correct and wrong classified images.

Task 31: Improvements

Think about ways to improve your model. You may, for example, try to use data augmentation (which is quite easy with Keras data loaders) or enlarge your training area (and thus the number of training samples). Further, you may increase or decrease the learning rate and the number of training epochs.

10 Become a practitioner with Data Mining tasks.

Task 32: Mining Approaches

Starting from the *LiveCoding* notebook and dataset (https://www.moodle.tum.de/course/section.php?id=3718743 apply different supervised and unsupervised techniques to create a model.

- Clustering
- Regression
- Classification
- Decision Tree
- ...

You can use other data and approaches from the lecture (see data mining notebooks).

Task 33: Model Evaluation

The evaluation of your models is a crucial step. Calculate and interpret evaluation metrics for your models (following https://www.moodle.tum.de/mod/resource/view.php?id=3246274).

Task 34: SQL*

Manage your patches in SQLite and make them accessible via API with 'fire' (https://github.com/google/python-fire).

11 Multilateration

"Pseudo-range multilateration, often simply multilateration when in context, is a technique for determining the position of an unknown point, such as a vehicle, based on measurement of the times of arrival of energy waves traveling between the unknown point and multiple stations at known locations." [Wikipedia]

Task 35: Multilateration

Write a program to perform multilateration. You should first review the lecture slides and familiarize yourself with the algorithm.

Following is a summary of the algorithm:

a. Set up a non-linear system of equations from measurements.

b. Create a linearized version locally by choosing an initial value and using Taylor expansion at this point.

c. Solve this equation and obtain an update vector.

d. Update this and go back to b. with this new place information

Thus, implement the following steps:

a. Calculate distances: $\alpha_i = \sqrt{(x_i - \tilde{x})^2 + (y_i - \tilde{y})^2}$ for $i = 1, ..., 4$.

b. Build matrix: $A_i = (-\frac{(x_i - \tilde{x})}{\alpha_i}, -\frac{(y_i - \tilde{y})}{\alpha_i})$.

c. Create: $b_i = d_i - \alpha_i$.

d. Solve: $A \begin{pmatrix} \hat{x} \\ \hat{y} \end{pmatrix} = b$.

e. Compute: $\begin{pmatrix} \tilde{x}_2 \\ \tilde{y}_2 \end{pmatrix} = \begin{pmatrix} \tilde{x}_1 \\ \tilde{y}_1 \end{pmatrix} + \begin{pmatrix} \hat{x} \\ \hat{y} \end{pmatrix}$.

You can use the following data from the lecture:

Component	**Coordinate**	**Measurement** (d_j)
Base station 1	$(x_1, y_1) = (0, 0)$	2.92
Base station 2	$(x_2, y_2) = (10, 0)$	8.14
Base station 3	$(x_3, y_3) = (15, 10)$	15.46
Base station 4	$(x_4, y_4) = (0, 12)$	9.89
Mobile Device	$(x_e, y_e) = (2, 2)$	–
Initial Location Estimate	$(\tilde{x}, \tilde{y}) = (20, 20)$	–

12 Exam Preparation

In this tutorial, we practice a mock exam.

Task 36: State your Name

Name:

Surname:

Task 37: Yes-No-Example

Answer the following questions with yes or no.

Are the following statements true or false?		True	False
a	In Python, you need to define a function to execute a script.		
b	The Numpy and Matplotlib library are by default part of the Python installation.		
c	To install a Python package, you always have to create a virtual environment.		

Task 38: Multiple Joice Example

For the given question, one of the answer options is correct. Mark this (and only this) answer. If more than one answer is marked, the answer is not considered. A wrong answer is **not** counted as -1.

In programming, what is meant by a recursive function?		
a	A function that is called multiple times.	
b	A function that calls itself.	
c	A function inside another function.	
d	A function that takes another function as input.	

Which of the following is **not** a method of a list object in Python?		
a	remove	
b	delete	
c	reverse	
d	pop	

Task 39: Distance Between Two Points

Complete the given Python function `dist()` that takes two points `p1 = [x1, y1]` and `p2 = [x2, y2]` as input and shall return their distance.

```
1    def dist(p1, p2):
2        # ...
3        return distance
```

Task 40: List Comprehension

a. Consider the following program using list expressions.

```
1  vocabulary = ['dear', 'done', 'easy', 'hello', 'hope', 'is', 'it', 'say
       ', 'student', 'to', 'we', 'well']
2  sentence = " ".join([vocabulary[x] for x in [3,0,8,10,4,6,5,2]])
3  print(sentence)
```

Give the exact output of the program

b. Consider the following Python function consisting of seven lines of code.

```
1  def myFunction(n: list):
2      result = []
3      for k in n:
4          if k > 0:
5              tmp = k**2
6              result.append(tmp)
7      return result
```

Rewrite the function with a list comprehension such that it only needs two lines of code.